

Qlawie Airlines uitleg voor partner

Dit document legt het project simpel uit. Het doel is dat iemand die weinig weet van PHP en Tailwind toch kan begrijpen hoe de website werkt.

Wat is dit project?

Qlawie Airlines is een boekingswebsite.

Gebruikers kunnen:

- reizen bekijken
- losse vluchten bekijken
- een reis of vlucht boeken
- inloggen en registreren
- hun boekingen bekijken in hun account
- recensies plaatsen
- contact opnemen
- wachtwoord resetten

Beheerders kunnen in het admin panel:

- reizen beheren
- vluchten beheren
- boekingen bekijken en status aanpassen
- berichten bekijken
- recensies goedkeuren of afkeuren

Belangrijkste technieken

Het project gebruikt:

- PHP voor de logica
- MySQL voor de database
- PDO voor database queries
- Tailwind CSS voor styling
- HTML formulieren voor invoer
- simpele JavaScript voor kleine interacties

Er wordt geen Laravel, React of ander framework gebruikt.

Mappenstructuur

public

Hier staan de pagina's die je in de browser bezoekt.

Voorbeelden:

- index.php
- reizen.php
- vluchten.php
- boeken.php
- account.php
- contact.php
- admin/index.php

Als je in de browser naar een pagina gaat, kom je meestal in deze map terecht.

app/includes

Hier staan onderdelen die op meerdere pagina's gebruikt worden.

Voorbeelden:

- navbar.php
- footer.php

De navbar en footer worden op veel pagina's ingeladen met:

```
include __DIR__ . "../app/includes/navbar.php";
```

app/page-includes/laden

Hier komen bestanden die data klaarzetten voor een pagina.

Voorbeeld:

laad-account.php hoort bij public/account.php.

In zo'n bestand haal je bijvoorbeeld de ingelogde gebruiker en boekingen op.

app/page-includes/ophalen

Hier komen bestanden die specifieke data ophalen.

Voorbeelden:

- haal-reizen-op.php hoort bij public/reizen.php
- haal-vluchten-op.php hoort bij public/vluchten.php
- haal-reisinformatie-op.php hoort bij public/reis-info.php
- haal-vluchtinformatie-op.php hoort bij public/vlucht-info.php

app/page-includes/verwerken

Hier staan bestanden die formulieren verwerken.

Voorbeelden:

- verwerk-inloggen.php
- verwerk-registratie.php
- verwerk-boeking.php
- verwerk-contactformulier.php
- verwerk-admin-reis.php
- verwerk-admin-vlucht.php

Een formulier stuurt data naar zo'n bestand via action.

Voorbeeld:

```
<form action="../../../app/page-includes/verwerken/verwerk-boeking.php" method="post">
```

Hoe werkt een PHP pagina?

Een pagina bestaat meestal uit 3 delen:

1. Een laadbestand includen
2. Navbar includen
3. HTML tonen met PHP variabelen

Voorbeeld:

```
<?php
include __DIR__ . "../../../app/page-includes/laden/laad-account.php";
include __DIR__ . "../../../app/includes/navbar.php";
?>
```

Daarna gebruikt de pagina variabelen die in het laadbestand zijn gemaakt.

Voorbeeld:

```
<?= $gebruiker['voornaam'] ?>
```

Dit toont de voornaam van de ingelogde gebruiker.

Belangrijke PHP begrippen

include

Met `include` laad je een ander PHP bestand in.

```
include __DIR__ . "../app/includes/navbar.php";
```

Dit betekent: plaats de code van `navbar.php` op deze plek.

require_once

Met `require_once` laad je een bestand dat echt nodig is.

Bijvoorbeeld de databaseverbinding:

```
require_once __DIR__ . "../../config/pdo.php";
```

Als dit bestand ontbreekt, stopt de pagina.

\$_GET

`$_GET` gebruik je om iets uit de URL te halen.

Voorbeeld URL:

```
vlucht-info.php?id=3
```

Dan haal je de id zo op:

```
$id = $_GET['id'] ?? '';
```

Daarna kun je met die id de juiste vlucht ophalen.

\$_POST

`$_POST` gebruik je bij formulieren met:

```
method="post"
```

Voorbeeld formulier:

```
<input type="text" name="email">
```

In PHP pak je die waarde zo:

```
$email = $_POST['email'] ?? '';
```

\$_SESSION

\$_SESSION bewaart gegevens zolang de gebruiker ingelogd is.

Voorbeeld:

```
$_SESSION['gebruiker_id'] = $gebruiker['id'];
```

Daarmee weet de website welke gebruiker is ingelogd.

Voordat je sessions gebruikt, moet je dit doen:

```
session_start();
```

Databaseverbinding

De databaseverbinding staat in:

```
app/config/pdo.php
```

Daarin wordt \$pdo aangemaakt.

Die \$pdo gebruik je om queries uit te voeren.

Voorbeeld:

```
$stmt = $pdo->prepare("SELECT * FROM gebruikers WHERE id = :id");  
$stmt->bindParam(":id", $id);  
$stmt->execute();  
$gebruiker = $stmt->fetch();
```

prepare, bindParam, execute

Dit is de normale manier waarop queries worden uitgevoerd.

```
$stmt = $pdo->prepare("SELECT * FROM gebruikers WHERE email = :email");  
$stmt->bindParam(":email", $email);  
$stmt->execute();
```

Betekenis:

- `prepare` maakt de query klaar
- `bindParam` vult een veilige waarde in
- `execute` voert de query uit

Dit voorkomt SQL injection.

fetch() en fetchAll()

fetch()

Gebruik je als je 1 rij verwacht.

Voorbeeld:

```
$gebruiker = $stmt->fetch();
```

Daarna gebruik je:

```
$gebruiker['email']
```

fetchAll()

Gebruik je als je meerdere rijen verwacht.

Voorbeeld:

```
$boekingen = $stmt->fetchAll();
```

Daarna loop je erdoorheen:

```
foreach ($boekingen as $booking) {  
    echo $booking['boekingsnummer'];  
}
```

Belangrijke database tabellen

gebruikers

Hier staan accounts.

Belangrijke kolommen:

- id
- voornaam
- achternaam
- email
- telefoon
- wachtwoord_hash
- rol

reizen

Hier staan pakketreizen.

Belangrijke kolommen:

- id
- titel
- korte_beschrijving
- beschrijving
- vertrek_luchthaven
- aankomst_luchthaven
- vertrekdatum
- terugkomstdatum
- duur_dagen
- reisklasse
- prijs_vanaf
- beschikbare_plekken
- bagage_inbegrepen
- afbeelding

vluchten

Hier staan losse vluchten.

Belangrijke kolommen:

- id
- vertrek_luchthaven
- aankomst_luchthaven
- vertrek_datum
- aankomst_datum
- prijs
- stoelen
- vlucht_nummer

boekingen

Hier staan gemaakte boekingen.

Belangrijke kolommen:

- id

- boekingsnummer
- gebruiker_id
- reis_id
- vlucht_id
- klant_naam
- klant_email
- klant_telefoon
- aantal_reizigers
- reisklasse
- bagage_keuze
- totaalprijs
- status
- opmerkingen

Een boeking kan een reis of een vlucht hebben.

Bij een pakketreis:

boekingen.reis_id verwijst naar reizen.id

Bij een losse vlucht:

boekingen.vlucht_id verwijst naar vluchten.id

recensies

Hier staan reviews van gebruikers.

Belangrijke kolommen:

- id
- reis_id
- gebruiker_id
- naam
- rating
- bericht
- status

contact_berichten

Hier staan berichten van het contactformulier.

Belangrijke kolommen:

- id
- naam
- email
- onderwerp
- bericht
- status

wachtwoord_resets

Hier staan tijdelijke reset tokens.

Belangrijke kolommen:

- id
- gebruiker_id
- token_hash
- verloopt_op
- gebruikt_op

Wat is een LEFT JOIN?

Een LEFT JOIN gebruik je als je data uit meerdere tabellen nodig hebt.

Voorbeeld:

```
$stmt = $pdo->prepare("SELECT boekingen.*, reizen.titel FROM boekingen LEFT JOIN reizen  
ON boekingen.reis_id = reizen.id WHERE boekingen.gebruiker_id = :gebruiker_id");
```

Betekenis:

- pak boekingen
- kijk naar boekingen.reis_id
- zoek de reis waar reizen.id hetzelfde is
- voeg reizen.titel toe aan het resultaat

Daarna kun je dit gebruiken:

```
$boeking['titel']
```

Waarom niet altijd reizen.* gebruiken?

Dit kan verwarrend worden:

```
SELECT boekingen.*, reizen.*, vluchten.*
```

Want meerdere tabellen hebben dezelfde kolomnaam, bijvoorbeeld:

- boekingen.id
- reizen.id
- vluchten.id

PHP maakt daar allemaal `$boeking['id']` van.

Dan weet je niet altijd zeker welke id je krijgt.

Beter is:

```
SELECT boekingen.*, reizen.titel, vluchten.vlucht_nummer
```

Dan blijft `$boeking['id']` gewoon de id van de boeking.

Hoe werkt boeken?

De pagina `public/boeken.php` toont een formulier.

Daar kies je:

- een pakketreis
- of een losse vlucht

Het formulier stuurt naar:

```
app/page-includes/verwerken/verwerk-boeking.php
```

Daar gebeurt dit:

1. formulierdata wordt opgehaald met `$_POST`
2. er wordt gecontroleerd of naam en email ingevuld zijn
3. er wordt gecontroleerd of reis of vlucht gekozen is
4. prijs wordt opgehaald uit `reizen` of `vluchten`
5. totaalprijs wordt berekend
6. boeking wordt opgeslagen in `boekingen`

Hoe werkt inloggen?

Het loginformulier stuurt naar:

```
app/page-includes/verwerken/verwerk-inloggen.php
```

Daar wordt:

1. email opgehaald uit `$_POST`
2. gebruiker gezocht in database
3. wachtwoord gecontroleerd
4. gebruiker id opgeslagen in `$_SESSION`
5. gebruiker doorgestuurd naar account of admin

Hoe werkt het admin panel?

De admin pagina is:

`public/admin/index.php`

Die pagina gebruikt losse includes.

Voorbeelden:

- `public/admin/includes/statistieken.php`
- `public/admin/includes/reizen-tabel.php`
- `public/admin/includes/reis-formulier.php`
- `public/admin/includes/vluchten-tabel.php`
- `public/admin/includes/vlucht-formulier.php`
- `public/admin/includes/boekingen.php`
- `public/admin/includes/reviews.php`
- `public/admin/includes/berichten.php`

Zo blijft `admin/index.php` kleiner en overzichtelijker.

Hoe werkt wijzigen in admin?

Bij reizen en vluchten staat een wijzig-link.

Voorbeeld:

`index.php?reis_id=2#formulier`

Betekenis:

- `reis_id=2` geeft door welke reis gewijzigd moet worden
- `#formulier` springt naar het formulier op de pagina

In het laadbestand wordt met `$_GET['reis_id']` de juiste reis opgehaald.

Daarna worden de waarden in het formulier gezet.

Wat betekent #formulier?

Dit is geen PHP.

Dit is een HTML anchor.

Voorbeeld:

```
index.php?reis_id=2#formulier
```

De browser springt dan naar:

```
<article id="formulier">
```

Tailwind CSS uitleg

Tailwind gebruikt classes direct in HTML.

Voorbeeld:

```
<button class="rounded-md bg-accent px-4 py-2 text-white hover:bg-black">
```

Betekenis:

- `rounded-md` maakt hoeken rond
- `bg-accent` geeft de accentkleur als achtergrond
- `px-4` geeft horizontale padding
- `py-2` geeft verticale padding
- `text-white` maakt tekst wit
- `hover:bg-black` maakt achtergrond zwart als je eroverheen gaat

Veelgebruikte Tailwind classes

Layout

```
flex  
grid  
gap-4  
items-center  
justify-between  
mx-auto  
max-w-6xl
```

Ruimte

```
p-4  
p-5  
px-6
```

py-10
mt-4
mb-4

Tekst

text-sm
text-xs
font-bold
font-semibold
text-black
text-white
text-accent

Achtergrond en borders

bg-white
bg-black
bg-accent
border
border-black
rounded-md

Responsive classes

Tailwind gebruikt prefixes zoals:

md:
lg:
xl:

Voorbeeld:

```
<div class="grid gap-5 md:grid-cols-3">
```

Betekenis:

- op mobiel: normale grid onder elkaar
- vanaf medium scherm: 3 kolommen

Hoe herken je een formulier?

Een formulier ziet er zo uit:

```
<form action="../app/page-includes/verwerken/verwerk-contactformulier.php" method="post">
```

Belangrijk:

- `action` zegt naar welk bestand de data gaat
- `method="post"` betekent dat PHP het ophaalt met `$_POST`

- elke input heeft een `name`

Voorbeeld:

```
<input type="email" name="email">
```

In PHP:

```
$email = $_POST['email'] ?? '';
```

Wachtwoord reset met token

Bij wachtwoord vergeten wordt een reset token gemaakt.

Normaal zou die link via email gestuurd worden.

De link ziet ongeveer zo uit:

```
wachtwoord-resetten.php?token=abc123
```

De pagina haalt de token uit de URL:

```
$token = $_GET['token'] ?? '';
```

Daarna kan gecontroleerd worden of de token klopt.

Waarom is dit nodig?

Zonder token zou iemand alleen met een emailadres een wachtwoord kunnen resetten.

Met token moet iemand eerst de resetlink hebben.

Belangrijke pagina's en hun laadbestand

public/index.php	app/page-includes/laden/laad-homepage.php
public/account.php	app/page-includes/laden/laad-account.php
public/admin/index.php	app/page-includes/laden/laad-admin-overzicht.php
public/boeken.php	app/page-includes/laden/laad-boekformulier.php
public/over-ons.php	app/page-includes/laden/laad-over-ons-pagina.php
public/wachtwoord-vergeten.php	app/page-includes/laden/laad-wachtwoord-vergeten.php
public/wachtwoord-resetten.php	app/page-includes/laden/laad-wachtwoord-resetten.php

Belangrijke pagina's en hun ophaalbestand

public/reizen.php	app/page-includes/ophalen/haal-reizen-op.php
public/reis-info.php	app/page-includes/ophalen/haal-reisinformatie-op.php
public/vluchten.php	app/page-includes/ophalen/haal-vluchten-op.php
public/vlucht-info.php	app/page-includes/ophalen/haal-vluchtinformatie-op.php
public/resultaten.php	app/page-includes/ophalen/haal-zoekresultaten-op.php

Simpele manier om een query te schrijven

Voor 1 rij:

```
$stmt = $pdo->prepare("SELECT * FROM gebruikers WHERE id = :id");  
$stmt->bindParam(":id", $id);  
$stmt->execute();  
$gebruiker = $stmt->fetch();
```

Voor meerdere rijen:

```
$stmt = $pdo->prepare("SELECT * FROM reizen ORDER BY titel");  
$stmt->execute();  
$reizen = $stmt->fetchAll();
```

Met meerdere voorwaarden:

```
$stmt = $pdo->prepare("SELECT COUNT(*) AS totaal FROM boekingen WHERE gebruiker_id = :  
gebruiker_id AND status = :status");  
$stmt->bindParam(":gebruiker_id", $gebruiker_id);  
$stmt->bindParam(":status", $status);  
$stmt->execute();  
$totaal = $stmt->fetch();
```

Veelgemaakte foutjes

`fetchAll()` gebruiken voor 1 gebruiker

Niet handig:

```
$gebruiker = $stmt->fetchAll();
```

Dan moet je doen:

```
$gebruiker[0]['email']
```

Beter:

```
$gebruiker = $stmt->fetch();
```

Dan kun je doen:

```
$gebruiker['email']
```

Verkeerde kolom in WHERE

Fout:

```
WHERE id = :gebruiker_id
```

Als je boekingen van een gebruiker zoekt, moet het zijn:

```
WHERE gebruiker_id = :gebruiker_id
```

Placeholder vergeten

Fout:

```
WHERE status = bevestigd
```

Beter:

```
WHERE status = :status
```

En daarna:

```
$status = "bevestigd";  
$stmt->bindParam(":status", $status);
```

Korte samenvatting

De `public` pagina toont HTML.

De `laden` en `ophalen` bestanden halen data uit de database.

De `verwerken` bestanden verwerken formulieren.

Tailwind is alleen styling via classes.

PDO queries halen data op uit MySQL.

`fetch()` is voor 1 rij.

`fetchAll()` is voor meerdere rijen.

`$_GET` is voor data uit de URL.

`$_POST` is voor data uit formulieren.

`$_SESSION` is voor ingelogde gebruikers.